

TypeScript_9_10

top

topタイプはシステムの中で可能なすべての値である。

このtopタイプと似ているタイプはany, unknown二つである。

```
0.3  Run  Export  Share
1  let a: any;
2  a = "abc";
3  a = 123
4
5  console.log(a);
```

[LOG]: 123

まずanyはすべてのタイプの場所で提供されることがtopとにている。

```
function greetComedian(name: unknown) {
  console.log(`Announcing ${name.toUpperCase()}!`);
}
```

Errors in code

'name' is of type 'unknown'.

unknownはanyとにているが、タイプの値を制限的することが違う。

keyof

```
interface me {
  id: number;
  name: string;
  age: number;
}

type Keys = keyof me;

let myProperty: Keys;

myProperty = "name";
myProperty = "age";
myProperty = "abc";
```

Errors in code

Type '"abc"' is not assignable to type 'keyof me'.

keyofはオブジェクトタイプのすべてのkeyを抽出することである。

簡単に言うとclassとかinterfaceのプロパティを名前だけ抽出してタイプに作る時に使用する。

type assertion

タイプをシステムが分かることができない場合もあるのでシステムに知らせるために

type assertionを使用する。

```
const rawData = '["grace", "frankie"]';

// タイプ : any
JSON.parse(rawData);

// タイプ : string[]
JSON.parse(rawData) as string[];

// タイプ : [string, string]
JSON.parse(rawData) as [string, string];

// タイプ : ["grace", "frankie"]
JSON.parse(rawData) as ["grace", "frankie"];
```

このようにasを付けて使用する。

```
try {
  // error code
} catch(error) {
  console.warn((error as Error).message);
}
```

このようにerrorのタイプを固定してErrorをみることもできる。

Generic

```
TS Playground v6.0.3 Run Export Share JS .D.TS Error

1 function abc<T>(a: T) {
2   return a;
3 }
4
5 const numeric = abc("abc");
6
7 const stringy = abc(123);
8
9 console.log("numeric type : ", typeof numeric);
10
11 console.log("stringy type : ", typeof stringy);
```

```
[LOG]: "numeric type : ", "string"
[LOG]: "stringy type : ", "number"
```

Typescriptにはすべてのタイプを処理することができるanyタイプがある。

しかし、anyタイプを使ってすべてのエラーを逃すと問題になる。

それで、Genericを使う。

Generic interface, class

```
interface Me<T> {  
  title: string;  
  info: T;  
}  
  
interface Yungihak {  
  name: string;  
  age: number;  
  gender: string;  
}  
  
const myProfile: Me<Yungihak> = {  
  title: "me",  
  info: {  
    name: "yungihak",  
    age: 29,  
    gender: "man"  
  }  
};
```

```

class Secret<Key, Value> {
  key: Key;
  value: Value;

  constructor(key: Key, value: Value) {
    this.key = key;
    this.value = value;
  }

  getValue(key: Key): Value | undefined {
    return this.key === key
      ? this.value
      : undefined;
  }
}

const storage = new Secret(12345, "luggage");

storage.getValue(1987)

```

このようにinterface, classを作ることにもできる。

Promise

Promiseは簡単に言うとリクエストをもらって結果を待つことである。

```

function Yungihak(): Promise<string> {
  return new Promise((resolve, reject) => {
    console.log("wait...");

    setTimeout(() => {
      const isSuccess = true;

      if (isSuccess) {
        resolve("resolve");
      } else {
        reject("reject");
      }
    }, 2000);
  });
}

```

```

Yungihak()
  .then((result) => {
    console.log(result);
  })
  .catch((error) => {
    console.error(error);
  })

```

コードをみると2秒ご isSuccess は true かどうかに対して Promise の状態を resolve, reject にきめる。